МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №5

по дисциплине «Вычислительная математика»

Тема: Решение нелинейных уравнений. Метод Ньютона.

Студент гр. 4342 Кринкин Н.К.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Вариант 8. $f(x) = 2x^2 - x^4 - 1 + ln(x)$

Изучить метод Ньютона для решения нелинейных уравнений. Найти корень заданного уравнения с заданной точностью, исследовать зависимость количества итераций от требуемой точности и оценить устойчивость метода к ошибкам в исходных данных.

Задание.

В лабораторной работе №5 предлагается, используя программы - функции NEWTON и Round из файла methods.cpp (файл заголовков methods.h), найти корень уравнения $f(x) = 0$ методом хорд с заданной точностью Eps, исследовать скорость сходимости и обусловленности метода.

Порядок выполнения работы должен быть следующим:

1) Графически или аналитически отделить корень уравнения $f(x) = 0$ (т.е. найти отрезки [Left, Right], на котором функция $f(x)$ удовлетворяет условиям сходимости метода Ньютона).

2) Составить подпрограммы - функции вычисления $f(x)$, $f'(x)$, предусмотрев округление их значений с заданной точностью Delta.

3) Составить головную программу, вычисляющую корень уравнения $f(x) = 0$ и содержащую обращение к подпрограммам $f(x)$, $f'(x)$, Round, NEWTON и индикацию результатов.

4) Выбрать начальное приближение корня x0 из [Left, Right] так, чтобы $f(x)f''(x) > 0$

5) Провести вычисления по программе. Исследовать скорость сходимости метода и чувствительность метода к ошибкам в исходных данных.

Основные теоретические положения.

В случае, когда известно хорошее начальное приближение решения уравнения $f(x) = 0$, эффективным методом повышения точности является метод Ньютона. Он состоит в построении итерационной последовательности $x_{n+1} = x_n - f(x_n)/f'(x_n)$ сходящейся к корню уравнения $f(x) = 0$.

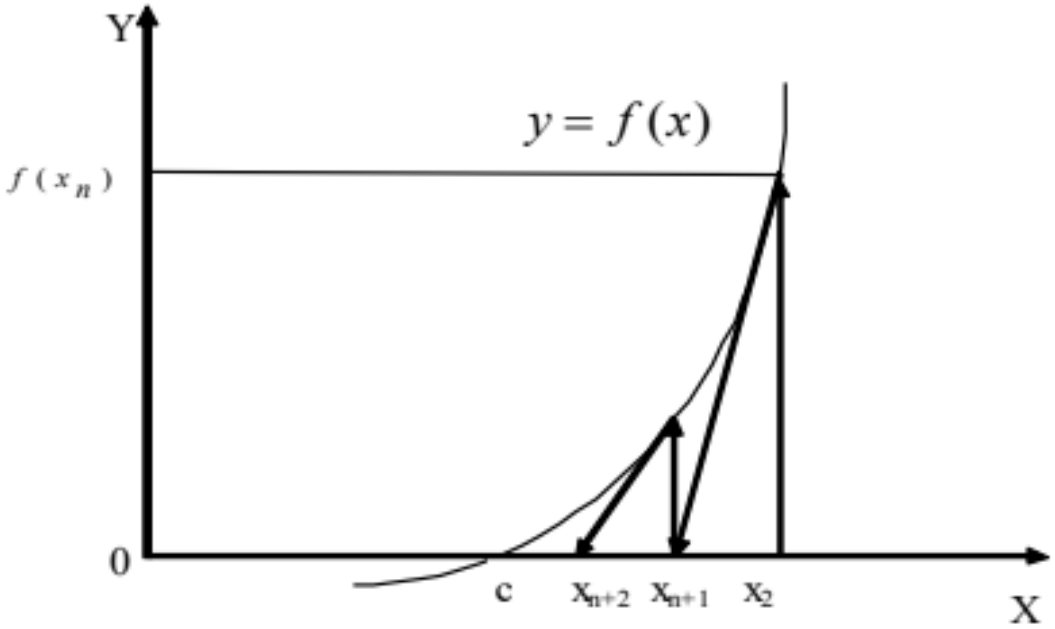

Рис. 1

Метод Ньютона допускает простую геометрическую интерпретацию (рис. 1). Если через точку с координатами $(x_n; f(x_n))$ провести касательную, то абсцисса точки пересечения этой касательной с осью Ох будет очередным приближением x_{n+1} корня уравнения.

Для оценки погрешности n-го приближения корня предлагается пользоваться неравенством

$$\left|c - x_n\right| \leq \frac{2M_2}{m_1}\left|x_n - x_{n-1}\right|^2$$

где M_2 – наибольшее значение модуля второй производной $|f''(x)|$ на отрезке [a,b]; m_1- наименьшее значение модуля первой производной $|f'(x)|$ на

отрезке [a,b]. Таким образом, если $\left|x_n - x_{n-1}\right| < \varepsilon$, то $\left|c - x_n\right| \leq \frac{2M_2\varepsilon^2}{m_1}$. Это означает, что при хорошем начальном приближении корня после каждой итерации число верных десятичных знаков в очередном приближении удваивается, т.е. процесс сходится очень быстро (имеет место квадратичная сходимость). Из указанного следует, что при необходимости нахождения корня с точностью итерационный процесс можно прекращать, когда

$$\left|x_n - x_{n-1}\right| < \varepsilon_0 = \sqrt{\frac{2m_1\varepsilon}{M_2}}$$

Рассмотрим один шаг итераций. Если на (n-1)-м шаге очередное приближение x_{n-1} не удовлетворяет условию окончания процесса, то вычисляются величины $f(x_{n-1})$, $f'(x_{n-1})$ и следующие приближение корня $x_n = x_{n-1} - f(x_{n-1})/f'(x_{n-1})$

Выполнение работы.

1) Нахождение отрезка

   Построив график функции (рис. 2), выберем промежуток [0, 2] в отправной точки. Как видно по графику, отрезок содержит корень уравнения. Возьмем $x_0 = 1.05$ в качестве начального приближения.

   Проверим корректность начального приближения с точки зрения обусловленности метода. Для этого вычислим первую и вторую производные функции и построим их графики (рис. 2):

   $$f'(x) =- 4x^3 + 4x - \frac{1}{x}$$

   $$f''(x) =- 12x^2 + 1/(x^2) + 4$$

   Вторая производная обращается в ноль на промежутке [0.7; 0.75], значение $f(x_0) < 0$, следовательно выполняется условие сходимости метода $f(x)f''(x) > 0$

2) Реализуем функцию f(x) и f’(x), используя инструменты стандартной библиотеки C++:

```
double F(double x) {
    return 2 * pow(x, 2) - pow(x, 4) - 1 - log(x);
}

double F1(double x) {
    return 4 * x - 4 * pow(x, 3) - pow(x, -1);
}
```

Полный код программы представлен в приложении А.

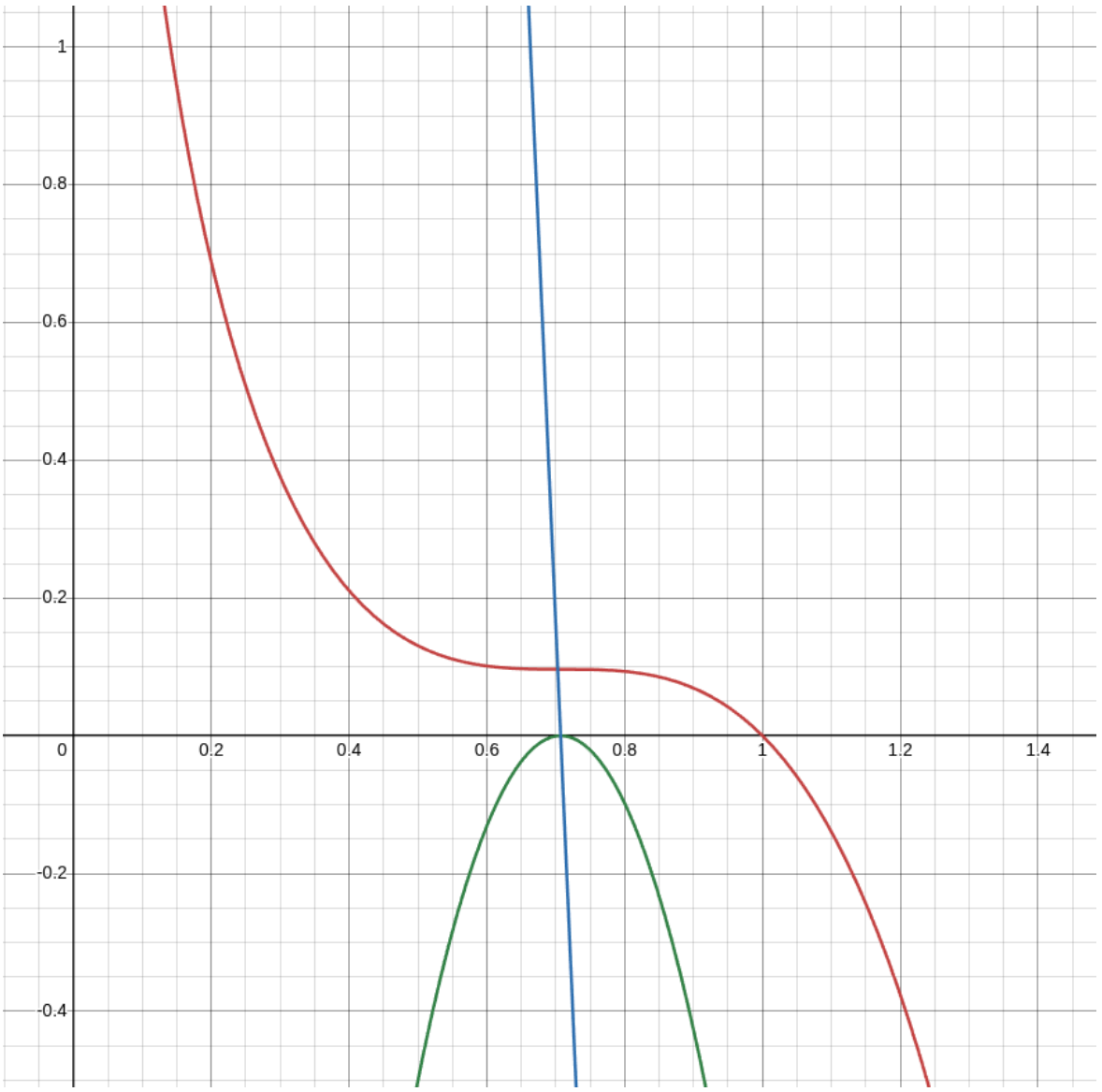

Рисунок 2 - график функции $f(x)$ (красный), ее первой производной $f'(x)$ (зеленый) и второй производной $f''(x)$ (синий)

3) В основной файле определим функцию main, которая представляет, которая представляет из себя цикл, перебирающий значение порядка точности ε и выводит результат работы функции NEWTON с точностью delta

4) Проведены вычисления, построен график (Рисунок 3).

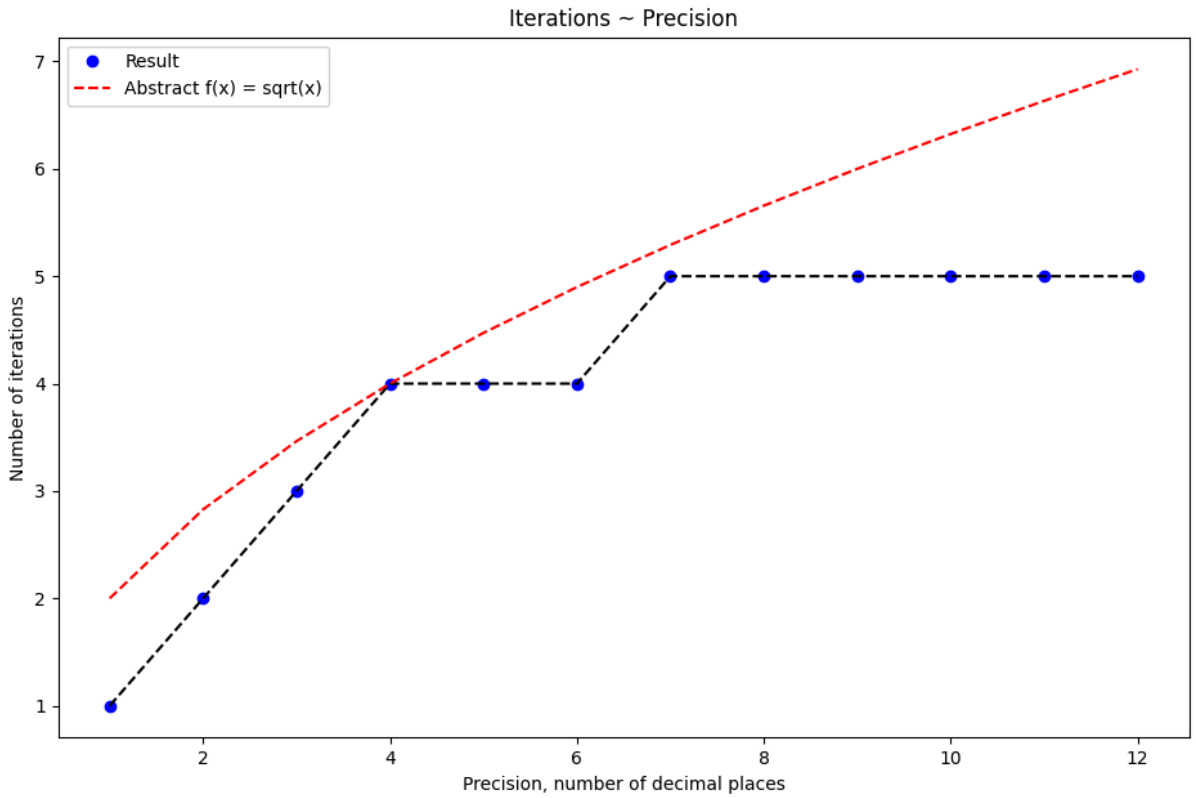

Рисунок 3. График зависимости количества итераций от порядка точности

Если провести исследование графика, можно понять, что порядок точность и количество итераций формируют зависимость квадратного корня, что является преимуществом и свойством метода Ньютона.

5) Для исследования чувствительности метода ко входным данным была написана дополнительная программа, проводящая итерацию по начальным приближениям в промежутке от [0.2; 2) с шагом 0.02 и измеряющая количество шагов алгоритма для нахождения корня. В рамках данного исследования была выбрана точность eps = 10^-8, чтобы влияние

ошибок входных данных или ошибок в выборе корня были более заметны. Результаты представлены на рисунке 4.

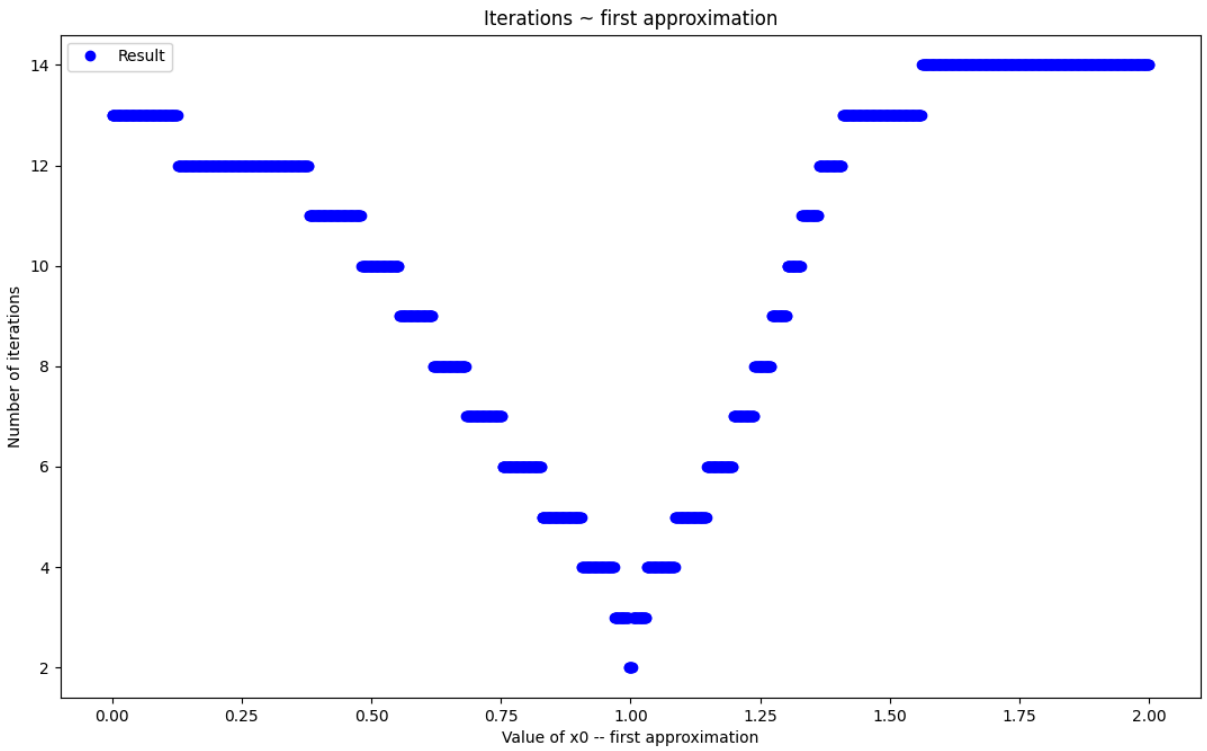

Рисунок 4. График зависимости количества итераций от начального приближения

Исходя из результатов исследования, можно сделать вывод об устойчивости метода отрезков к ошибкам в округлении, а также увидеть “значимость” хорошего стартового приближения.

Выводы.

В ходе работы был изучен метод простых итераций. Для уравнения $f(x) = 2x^2 - x^4 - 1 + ln(x)$ корень был успешно найден на отрезке [0.85; 1.15] с точностью от 0.1 до 10^-12. Построенный график показал корневую зависимость числа итераций от порядка точности: увеличение точности (уменьшение ε) требует ожидаемого роста количества вычислений, что соответствует теории. Исследование обусловленности показало, что метод Ньютона является устойчивым к малым ошибкам округления значений функции,

однако при требует корректного определения стартового приближения, иначе процесс может замедлиться или привести к потере точности нахождения корня.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл `precision.cpp`:

```
#include <iostream>
#include <cstdio>
#include <cmath>
#include <string>
#include <vector>

#define __NEWTON

#include "methods.h"

using namespace std;

double F(double x) {
    return 2 * pow(x, 2) - pow(x, 4) - 1 - log(x);
}

double F1(double x) {
    return 4 * x - 4 * pow(x, 3) - pow(x, -1);
}

int main(int argc, char *argv[]) {
    int n;
    double x;
    double eps;
    double delta = pow(10, -4);

    FILE *out_file = fopen(argv[1], "w");

    double x0 = 1.05;

    for (int i=1; i<=12; i++) {
        eps = pow(10, -i);
        x = NEWTON(x0, eps, n);
        fprintf(out_file, "%d %d\n", i, n);
    }
    fclose(out_file);

    return 0;
}
```

Файл `lab.cpp`:

```
#include <iostream>
#include <cstdio>
#include <cmath>
#include <string>
#include <vector>

#define __NEWTON
```

```
#include "methods.h"

using namespace std;

double F(double x) {
    return 2 * pow(x, 2) - pow(x, 4) - 1 - log(x);
}

double F1(double x) {
    return 4 * x - 4 * pow(x, 3) - pow(x, -1);
}

int main() {
    double x;
    double eps;
    double delta;

    int it;
    cout << "Enter epsilon presicion (decimal points): ";
    cin >> it; if (it <= 0) it = 4;
    eps = pow(10, -it);

    cout << "Enter delta presicion (decimal points): ";
    cin >> it; if (it <= 0) it = 4;
    delta = pow(10, -it);

    double x0 = 1.05;
    int n = 0;
    x = NEWTON(x0, eps, n);

     printf("\tFor epsilon = %lf and x0 = %lf: x = %lf with delta =
%lf, iter = %d\n", eps, x0, Round(x, delta), delta, n);

    return 0;
}
```

Файл `input_errors.cpp`:

```
#include <iostream>
#include <cstdio>
#include <cmath>
#include <string>
#include <vector>

#define __NEWTON

#include "methods.h"

using namespace std;

double F(double x) {
    return 2 * pow(x, 2) - pow(x, 4) - 1 - log(x);
}

double F1(double x) {
    return 4 * x - 4 * pow(x, 3) - pow(x, -1);
}
```

```
int main(int argc, char *argv[]) {
    int n;
    double x;
    double eps = pow(8, -10);

    FILE *out_file = fopen(argv[1], "w");

    for (double x0=0.2; x0 < 2; x0 += 0.001) {
        x = NEWTON(x0, eps, n);
        fprintf(out_file, "%lf %d\n", x0, n);
    }

    return 0;
}
```

Файл `graph.py`:

```
import matplotlib
matplotlib.use("TkAgg")

import os
import sys
from matplotlib import pyplot as plt


if __name__ == "__main__":
    data_x = []
    data_y = []


    if not os.path.exists(sys.argv[1]):
        raise FileNotFoundError(f"No {sys.argv[1]} file!")
    file_nm = sys.argv[1]
    file = open(file_nm, "r")

    if sys.argv[2] == "precision":
        for line in file.readlines():
            if line:
                x, y = line.split()
                data_x.append(int(x))
                data_y.append(int(y))

        plt.plot(data_x, data_y, "bo", label="Result")
        plt.plot(data_x, data_y, "k--")

              plt.plot(data_x,  list(map(lambda  x:  2  *  x**(1/2),
data_x)), "r--", label="Abstract f(x) = sqrt(x)")
        plt.title("Iterations ~ Precision")
        plt.xlabel("Precision, number of decimal places")
        plt.ylabel("Number of iterations")

    else:
        for line in file.readlines():
            if line:
                x, y = line.split()
                data_x.append(float(x))
```

```
                    data_y.append(int(y))

                  plt.axvline(x=0.70711,  color='red',  linestyle='--',
label='f\'(x) = 0 \nf\'\'(x) = 0')
             plt.plot(data_x, data_y, "bo", label="Result")
             # plt.plot(data_x, data_y, "k--")

             plt.title("Iterations ~ first approximation")
             plt.xlabel("Value of x0 -- first approximation")
             plt.ylabel("Number of iterations")

         plt.legend()
         plt.show()
```

Файл `Makefile`:

```
SRC_DIR = ${CURDIR}/src
WORK_DIR = ${SRC_DIR}/workdir
MAT_DIR = ${SRC_DIR}/Materials

lab: ${WORK_DIR}/lab
${WORK_DIR}/lab

precision: ${WORK_DIR}/precision.txt ${WORK_DIR}/graph.py
python3 ${WORK_DIR}/graph.py ${WORK_DIR}/precision.txt precision

input_errors: ${WORK_DIR}/input_errors.txt ${WORK_DIR}/graph.py
python3      ${WORK_DIR}/graph.py      ${WORK_DIR}/input_errors.txt
input_errors

${WORK_DIR}/precision.txt: ${WORK_DIR}/precision.o
${WORK_DIR}/precision.o ${WORK_DIR}/precision.txt

${WORK_DIR}/input_errors.txt: ${WORK_DIR}/input_errors.o
${WORK_DIR}/input_errors.o ${WORK_DIR}/input_errors.txt

${WORK_DIR}/lab: ${WORK_DIR}/lab.cpp ${MAT_DIR}
g++ -g -I${MAT_DIR} ${WORK_DIR}/lab.cpp -o ${WORK_DIR}/lab

${WORK_DIR}/precision.o: ${WORK_DIR}/precision.cpp ${MAT_DIR}
g++      -g      -I${MAT_DIR}      ${WORK_DIR}/precision.cpp      -o
${WORK_DIR}/precision.o

${WORK_DIR}/input_errors.o:            ${WORK_DIR}/input_errors.cpp
${MAT_DIR}
g++     -g     -I${MAT_DIR}     ${WORK_DIR}/input_errors.cpp     -o
${WORK_DIR}/input_errors.o

help:
@echo "Available options:"
@echo "    make [lab] -- calculate f(x) = 0 and show results"
@echo " make precision -- calculate f(x) for different epsilon for
given delta"
@echo "    make input_errors -- show input data sensitivity"
```